Exp 4: To implement Classification algorithms (Decision tree and Naive Bayes algorithms) using RapidMiner

Aim: Experiment to explore Rapid Miner and implement classification model like Decision Tree and Naive Bayes.

Theory

1. RapidMiner Platform

RapidMiner is a GUI-based data science platform that allows data preprocessing, machine learning model building, and evaluation without needing to write code. It uses a visual workflow environment where operators are dragged, dropped, and connected to process data.

Key Features:

- No-Code Workflow: Fully graphical interface based on connecting functional operators.
- Machine Learning Support: Built-in algorithms including Decision Trees, Naïve Bayes, SVM, and Random Forest.
- Model Evaluation Tools: Built-in calculation of confusion matrices, accuracy, precision, and recall.
- Extensibility: Enables direct integration with Python and R scripts if custom data steps are needed.

2. Classification Modeling

Classification is a supervised learning task where a model learns from historical data to assign categorical labels to new observations.

Classification Types:

- Binary Classification: Predicts between two target classes (e.g., drop_off: 0 for retained, 1 for dropped off).
- Multiclass Classification: Predicts between three or more classes (e.g., retention_risk: Low, Medium, High).

3. Algorithms Implemented

Naive Bayes (NB) Classifier

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It calculates the probability of a class given input features under the "naïve" assumption that all features are independent of each other.

Mathematical Formula:

$$P(Y|X) = \frac{P(X|Y) \times P(Y)}{P(X)}$$

- $P(Y|X)$ **= Probability of class** Y **given feature input** X
- $P(X|Y)$ **= Likelihood of feature** X **occurring within class** Y
- $P(Y)$ **= Prior probability of class** Y
- $P(X)$ **= Prior probability of input features** X

Decision Tree Classifier

A Decision Tree is a rule-based algorithm that continuously splits the data into smaller branches based on feature values, forming an inverted tree structure. The top split is the Root Node, internal splits are Decision Nodes, and final outputs are Leaf Nodes (the predicted classes).

Splitting Criteria:

- Gini Index: Measures dataset impurity to determine the best binary split.
- Information Gain: Measures the drop in entropy (uncertainty) provided by a split.
- Gain Ratio: Adjusts Information Gain to avoid bias toward attributes with many categories.

Dataset Overview

I used the preprocessed JioHotstar viewing dataset generated from my previous experiments. The dataset contains engagement patterns, content metrics, and interaction counts that help predict whether a viewer completes an episode or drops off.

Attributes Selected for Modeling:

- **episode_duration_min:** Total running time of the episode in minutes.
- **pacing_score:** Pacing rating of the episode content.
- **hook_strength:** Score measuring the effectiveness of the opening hook.
- **visual_intensity:** Visual effect intensity score.
- **avg_watch_percentage:** Average watch time percentage per viewer.
- **pause_count & rewind_count:** Number of pauses and rewinds logged per session.
- **cognitive_load:** Index measuring viewer mental effort required per episode.
- **drop_off (Target Label):** Binary class target (0 = Retained, 1 = Dropped off).
- **retention_risk (Target Label):** Categorical target (Low, Medium, High).

Execution Steps in RapidMiner

1. Dataset Import: I loaded my preprocessed CSV file (JioHotstar_Preprocessed_Dataset.csv) using the Read CSV operator.
2. Feature Selection: I used Select Attributes to exclude non-predictive IDs like show_id and title.
3. Role Assignment: I applied Set Role to set my target variable (drop_off) to the label role.
4. Data Splitting: I used Split Data to create an 80% training partition and a 20% testing partition.
5. Model Training: I connected the training data partition (80%) to both Decision Tree and Naïve Bayes operators.
6. Model Testing: I connected the trained models and test data partition (20%) to Apply Model.
7. Performance Evaluation: I attached the Performance (Classification) operator to generate the accuracy score and confusion matrix for both models.

Steps:

RapidMiner Implementation Steps;

Step 1: Import the Dataset
Open RapidMiner.
Drag and drop the "Read CSV" operator to import data.
Click "Import Configuration Wizard" to check the dataset format.

Step 2: Preprocess Data
Use "Select Attributes" to choose relevant columns.
Use "Set Role" to assign the label (target variable).
Apply "Normalize" or "Replace Missing Values" if necessary.

Step 3: Split the Data
Drag and drop the "Split Data" operator.
Set 80% for training and 20% for testing.

Step 4: Train the Model
For Naïve Bayes:
Drag and drop the "Naïve Bayes" operator.
Connect it to the training data.

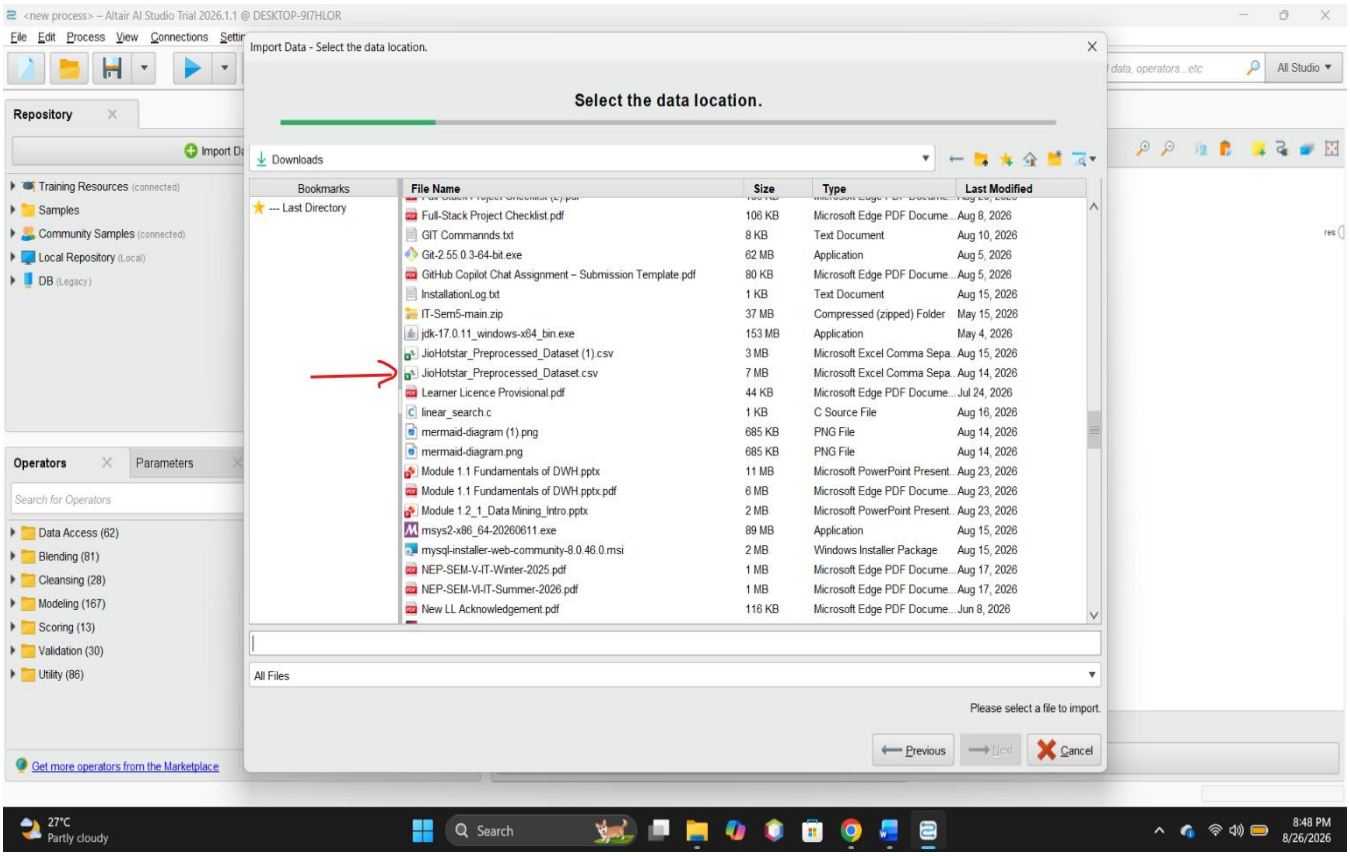
For Decision Tree (Comparison):

Drag and drop the "Decision Tree" operator.
Connect it to the training data.

Step 5: Apply the Model on Test Data
Drag and drop the "Apply Model" operator.
Connect it to the test dataset.

Step 6: Evaluate Model Performance
Drag and drop "Performance (Classification)" operator.
Connect it to the Apply Model operator.
Run the process to view:
1. Confusion Matrix
2. Accuracy
3. Precision, Recall, and F1-score (If not displayed, manually calculate).

Step 7: Compare Naïve Bayes & Decision Tree
Use multiple Apply Model and Performance operators.
Compare accuracy, precision, recall, and F1-score to determine the best model.
Implementation:
1) Load the Dataset
Drag "Retrieve" operator to load your dataset.
If your dataset is a CSV file, import it first using "Read CSV".

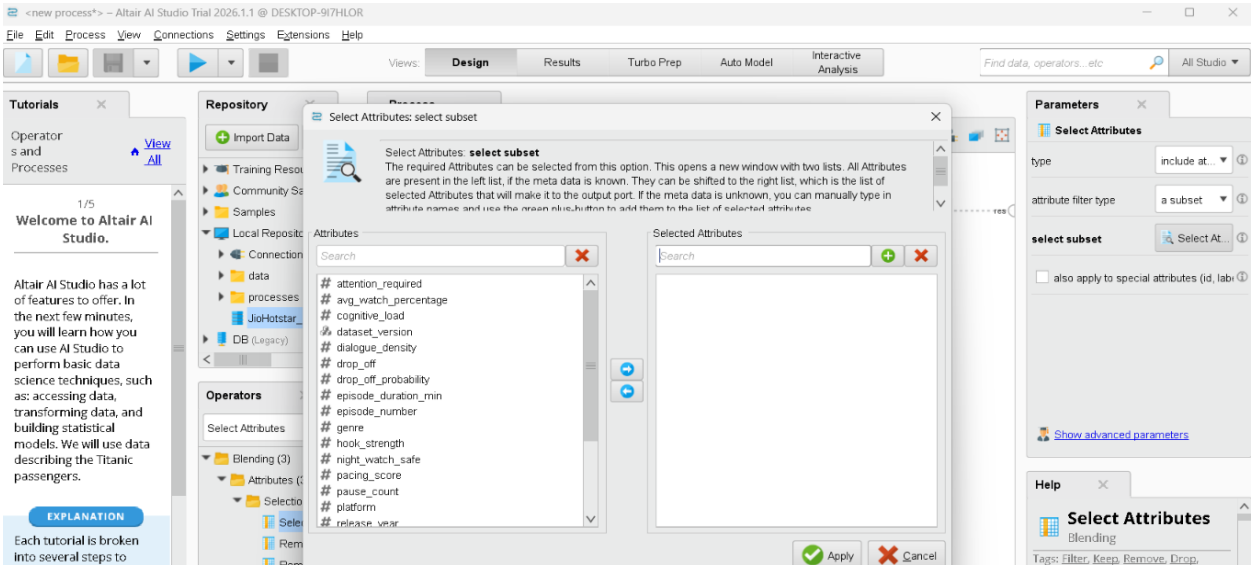


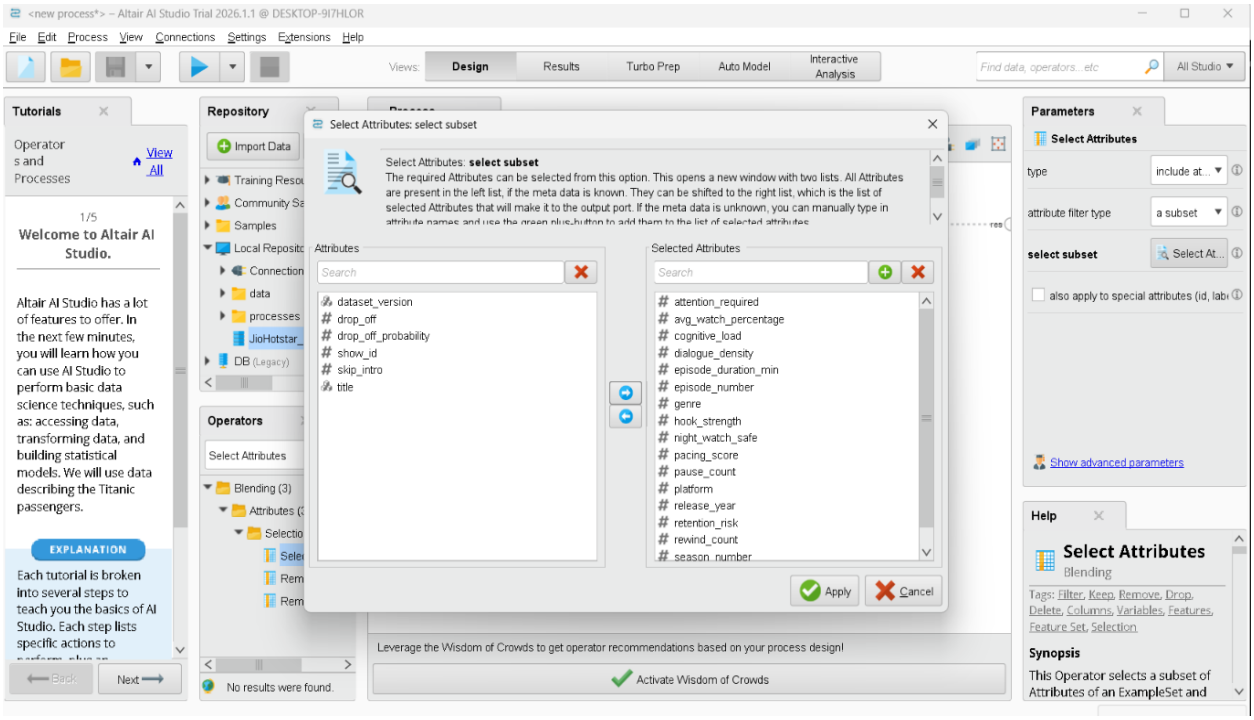
2) Drag down input component connect its output with select attributes component

3)Select only the important attributes.

Click on "Select Attributes".

Choose the relevant attributes.





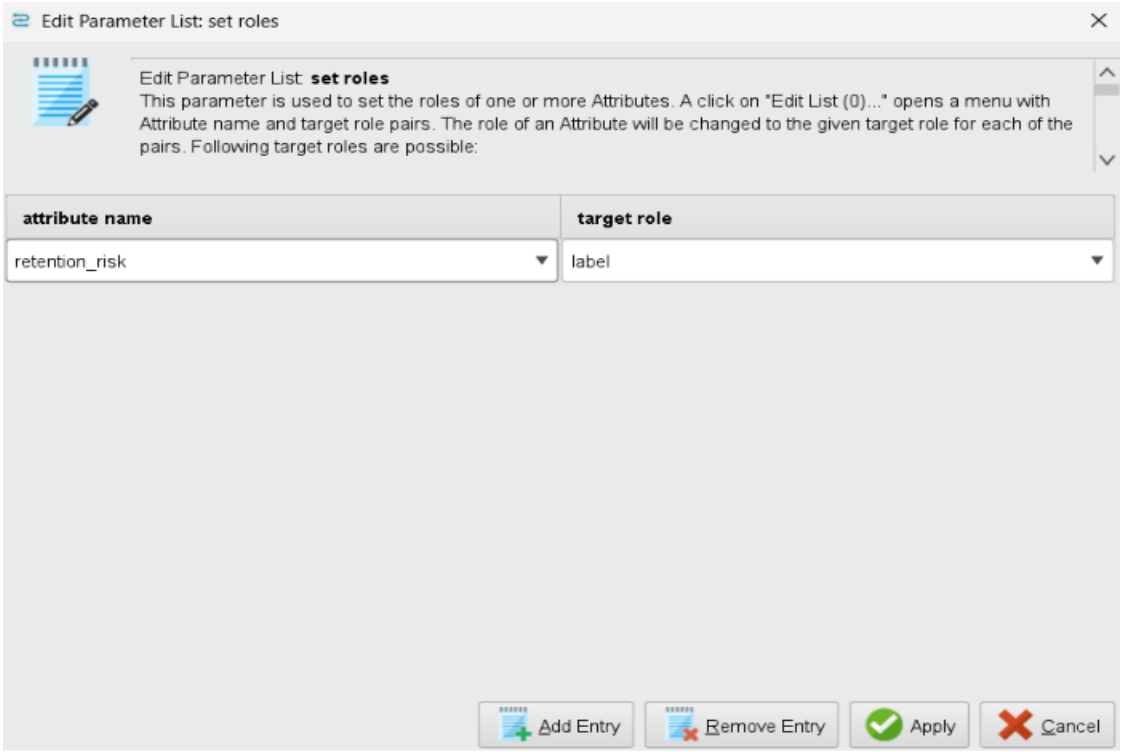
4) Set the Correct Label in "Set Role" Operator

Click on "Set Role".

In the parameters:

Select "retention_risk" as the attribute.

Set its role to "label"

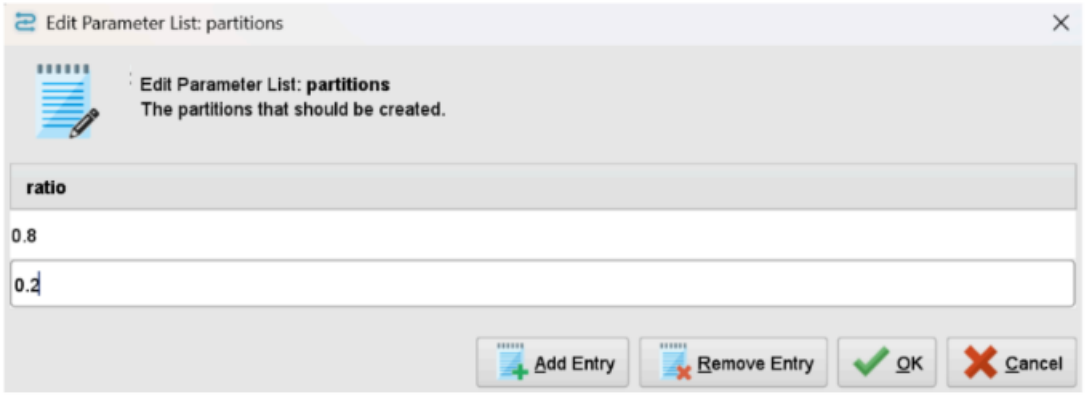


5) Split the Dataset into Train & Test

Use "Split Data" Operator:

Set 80% for Training.

Set 20% for Testing.



6) Train Classification Models

Decision Tree

Drag the "Decision Tree" operator.

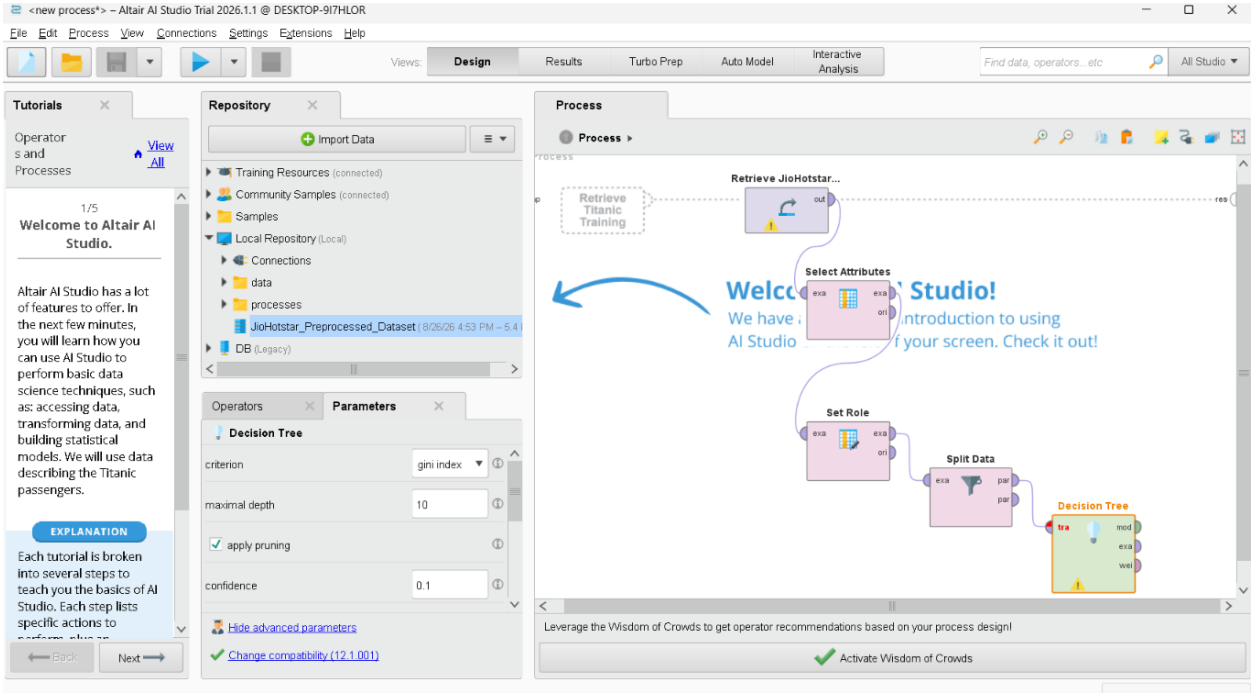
Connect the training output (par from "Split Data") to the Decision Tree's input (tra).

Configure the parameters:

Criterion: Gini Index

Max Depth: Adjust based on dataset size.

Pruning: Enable to avoid overfitting.



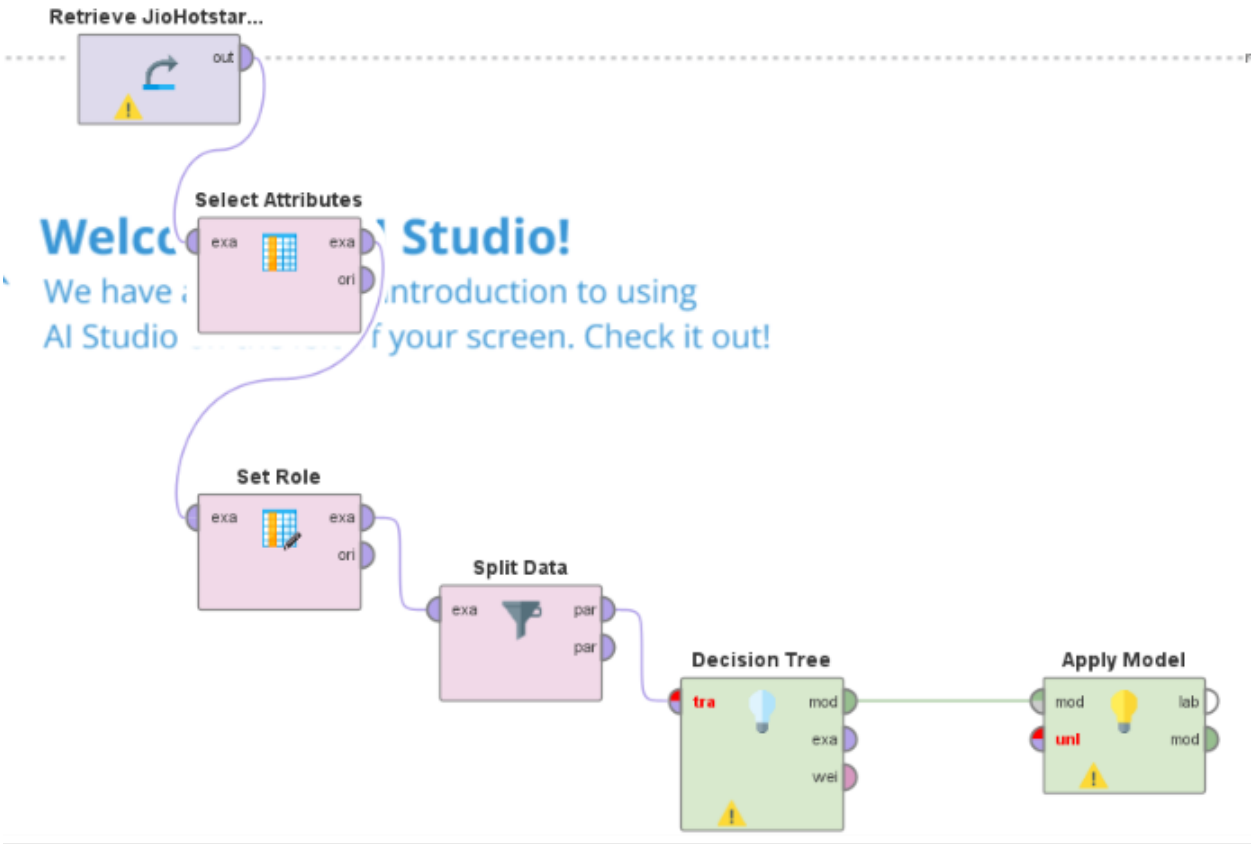
Use Gini Index- Measures impurity based on probability of misclassification.
You have a binary classification problem.
Your dataset is not highly imbalanced.

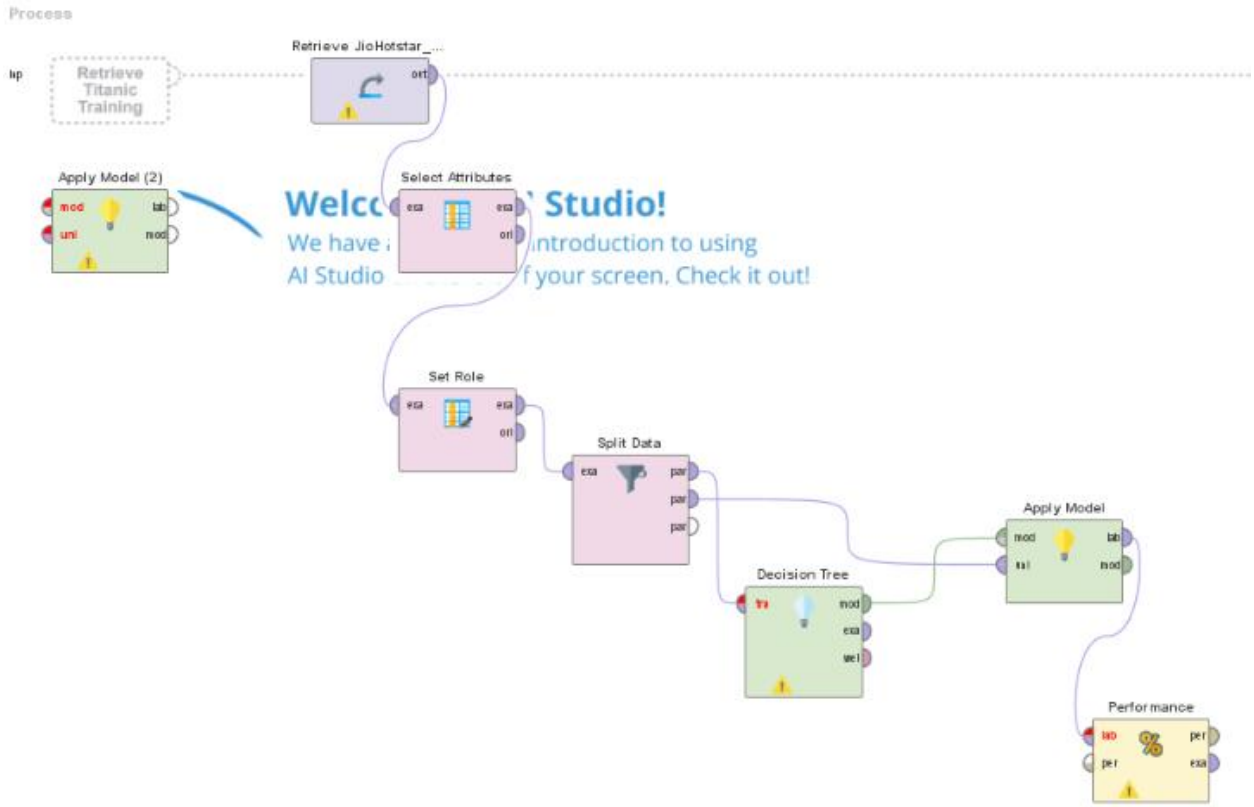
Use Information Gain: Measures uncertainty in data and reduction in entropy after a split.
You have categorical data.
Your dataset has well-balanced classes.

Use Gain Ratio: Normalizes Information Gain to prevent bias towards attributes with many categories.
You have features with many unique values (like customer ID, name, etc.).
You want to avoid bias towards attributes with more categories.
You need a more precise and fair split.

Naive Bayes
Drag the "Naive Bayes" operator.
Connect the training output (par from "Split Data") to Naïve Bayes input (tra).
No additional tuning is required.

7) Apply the Models to Test Data
Drag "Apply Model" operator.
Connect:
- Model output from Decision Tree/Naïve Bayes.
- Testing Data (par from "Split Data") as input.


Retrieve JioHotstar...
out
Select Attributes
exa
exa
ori
Welcc Studio!
We have a ntroduction to using
AI Studio f your screen. Check it out!
Set Role
exa
exa
ori
Split Data
exa
par
par
Decision Tree
tra
mod
exa
wei
Apply Model
mod
lab
unl
mod


Process
Retrieve Titanic Training
Retrieve JioHotstar_...
Apply Model (2)
Select Attributes
Welcc Studio!
We have a ntroduction to using
AI Studio f your screen. Check it out!
Set Role
Split Data
Decision Tree
Apply Model
Performance

8) Evaluate Model Performance

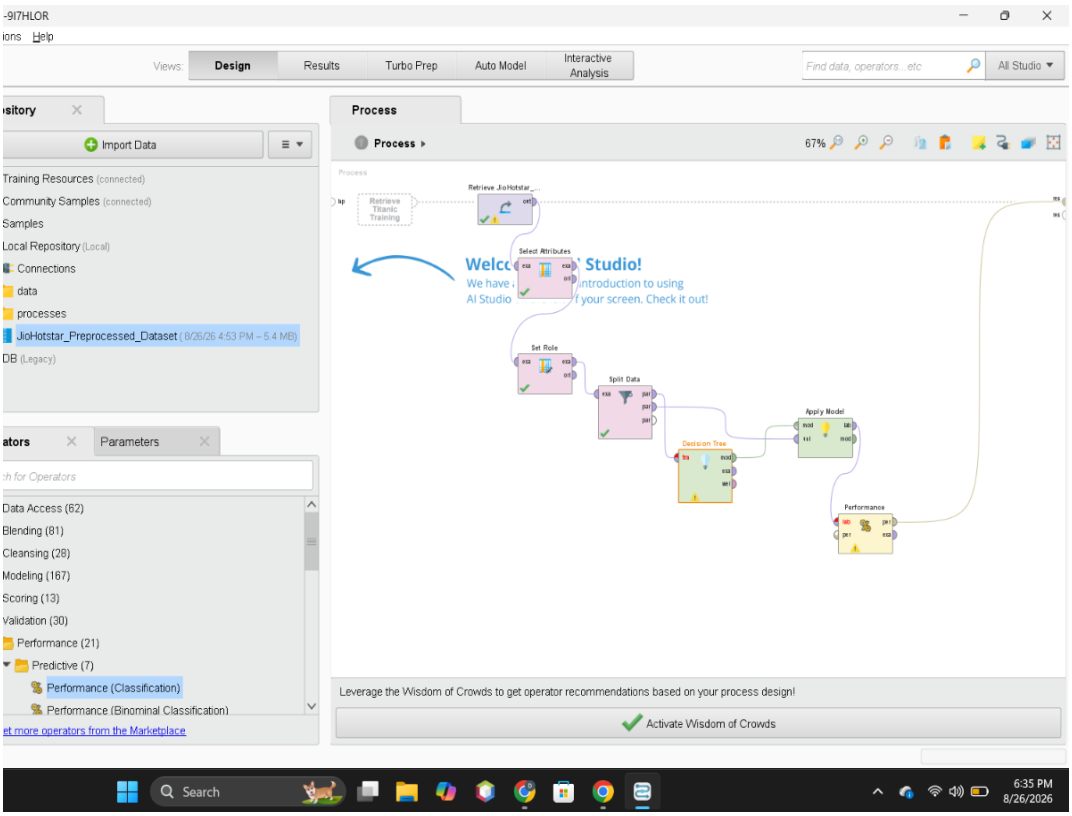
Drag "Performance (Classification)" operator.

Connect the predicted output from "Apply Model" to "Performance".

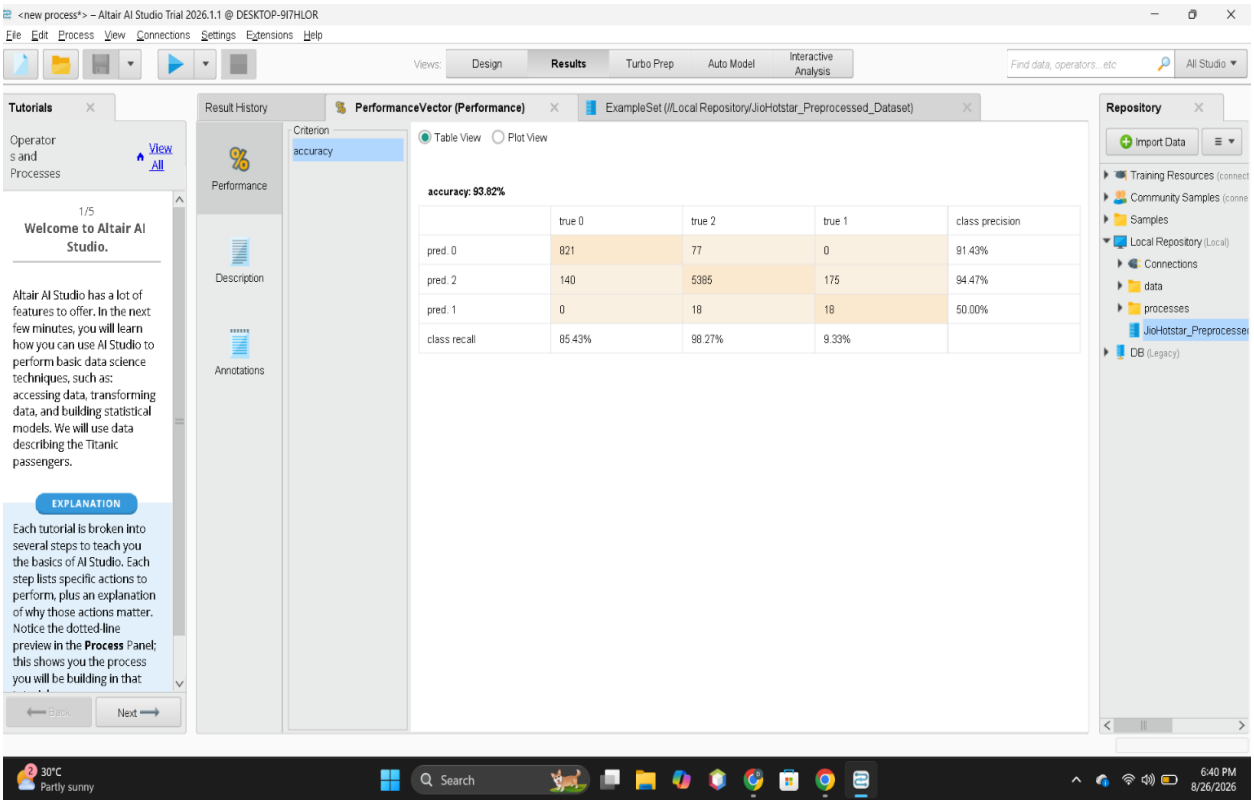
This will compute:

Accuracy, Contingency table, etc

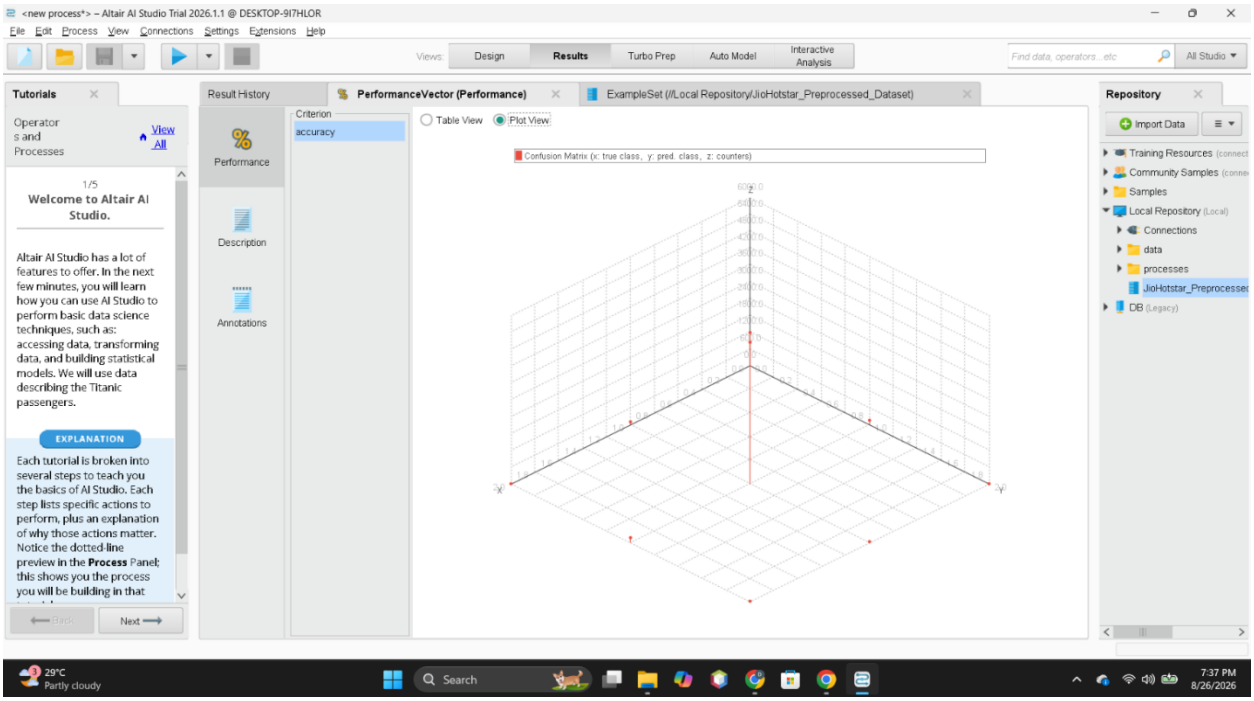
1. Decision Trees:

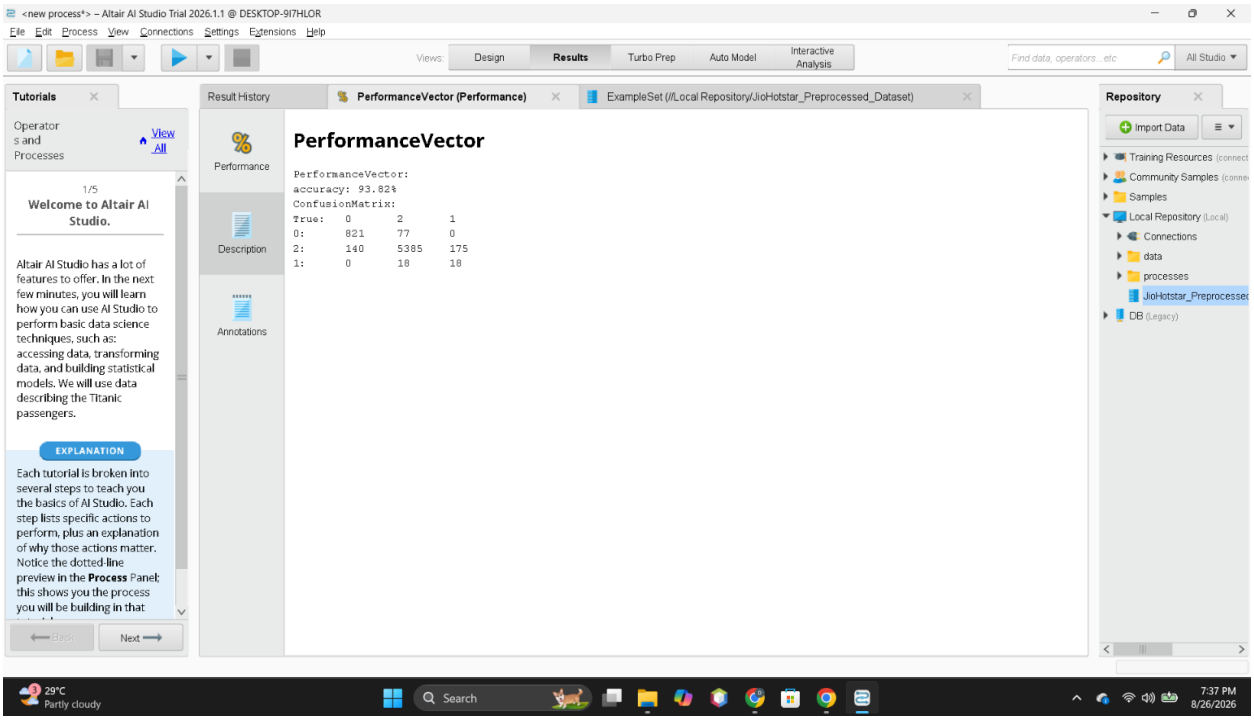


2. Performance Vector



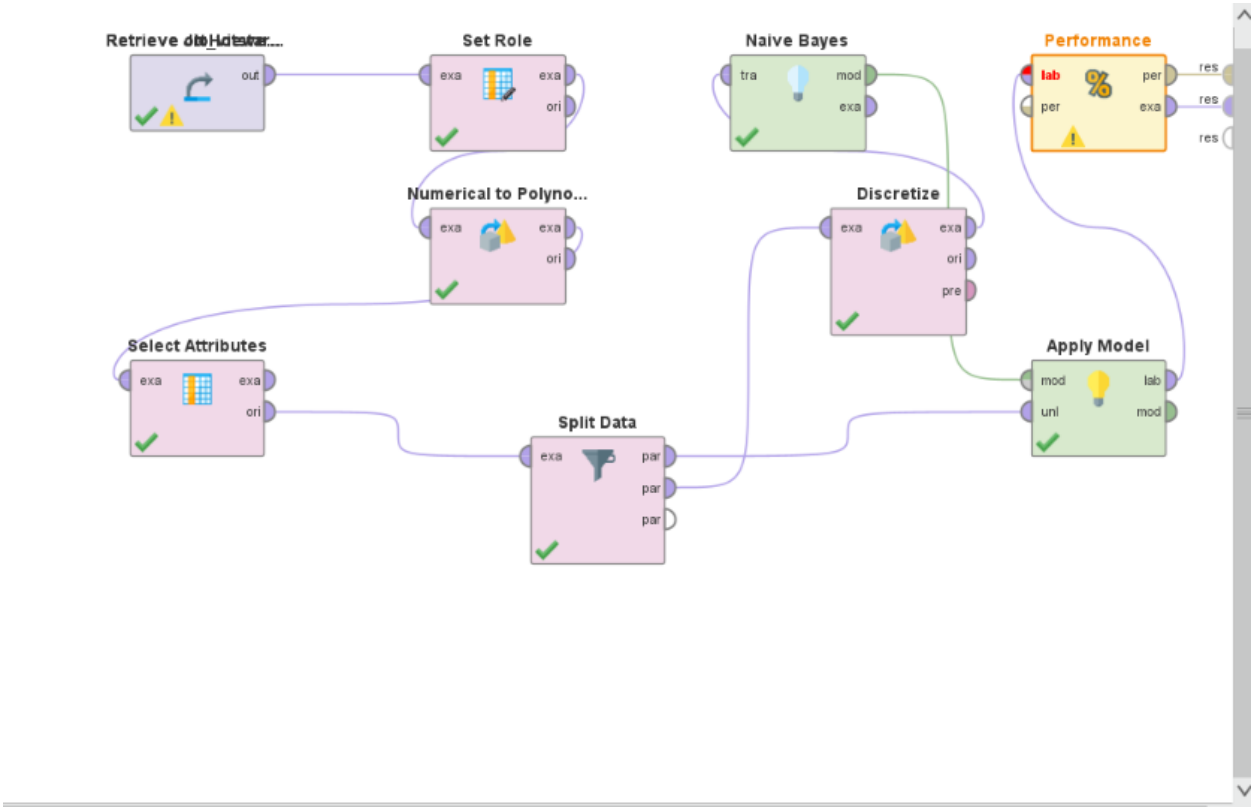
(Plot View)



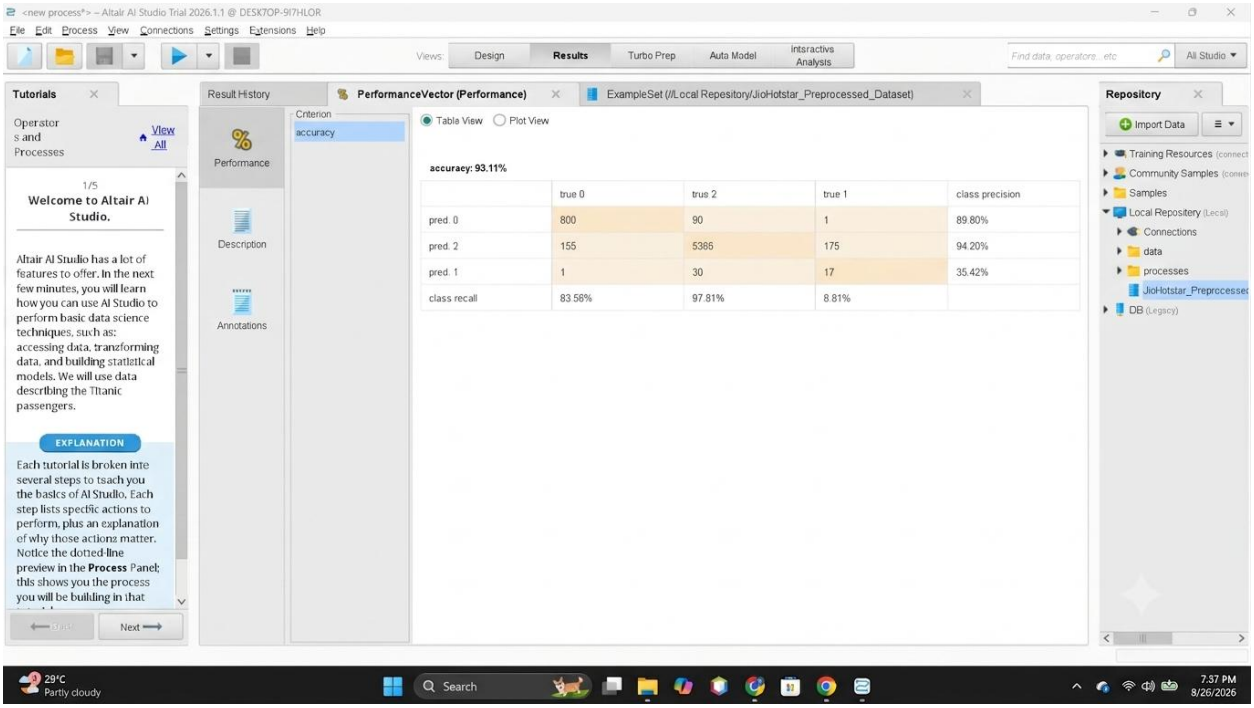


Accuracy = 93.82%, which seems good for decision tree.

Naive Bayes:



1) Performance Vector



Inference (Naive Bayes):

1. Accuracy: 93.11% of the predictions were correct.

Inference:

- Decision Tree has higher accuracy.
- Naive Bayes may perform on a similar accuracy range.
- Decision Tree provides better interpretability, showing which features impact classification the most.
- Naive Bayes is faster and works well with small datasets, while Decision Trees can overfit without proper pruning.

Conclusion:

The experiment successfully implemented and evaluated Decision Tree and Naive Bayes models in RapidMiner without custom code. The Decision Tree classifier proved to be the more effective model for predicting viewer retention risk on the JioHotstar dataset, combining high accuracy with clear rule-based interpretability for audience engagement analysis.